Công cụ - NumPy

*NumPy là thư viện nền tảng cho tính toán khoa học với Python. NumPy tập trung vào đối tượng mảng N-chiều mạnh mẽ, đồng thời chứa

Tạo mảng (Creating Arrays)

Bây giờ hãy import `numpy`. Hầu hết mọi người đều import nó với tên viết tắt là `np`:

```
import numpy as np
```

`np.zeros`

Hàm `zeros` tạo ra một mảng chứa bất kỳ số lượng số 0 nào:

```
np.zeros(5)
```

```
array([0., 0., 0., 0., 0.])
```

Việc tạo mảng 2 chiều (ví dụ: ma trận) cũng rất dễ dàng bằng cách cung cấp một tuple với số hàng và số cột mong muốn. Ví dụ, đây

```
np.zeros((3,4))
```

```
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```

Một số thuật ngữ (Vocabulary)

* Trong NumPy, mỗi chiều được gọi là một **axis**.
* Số lượng các trục (axis) được gọi là **rank**.
    * Ví dụ: ma trận 3x4 ở trên là một mảng rank 2 (nó có 2 chiều).
    * Trục đầu tiên có độ dài 3, trục thứ hai có độ dài 4.
* Danh sách độ dài các trục của mảng được gọi là **shape** (hình dạng) của mảng.
    * Ví dụ: shape của ma trận trên là `(3, 4)`.
    * Rank bằng với độ dài của shape.
* **Size** của mảng là tổng số phần tử, là tích của độ dài tất cả các trục (ví dụ: 3*4=12).

```
a = np.zeros((3,4))
a
```

```
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```

```
a.shape
```

```
(3, 4)
```

```
a.ndim  # equal to len(a.shape)
```

```
2
```

```
a.size
```

```
12
```

Mảng N-chiều

Bạn cũng có thể tạo mảng N-chiều với rank tùy ý. Ví dụ, đây là mảng 3D (rank=3), với shape `(2,3,4)`:

```
np.zeros((2,3,4))
```

```
array([[[0., 0., 0., 0.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.]],

       [[0., 0., 0., 0.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.]]])
```

Kiểu của mảng

Các mảng NumPy có kiểu dữ liệu là `ndarray`:

```
type(np.zeros((3,4)))
```

```
numpy.ndarray
```

`np.ones`

Nhiều hàm NumPy khác cũng tạo ra các `ndarray`.

Đây là ma trận 3x4 chứa đầy các số 1:

```
np.ones((3,4))
```

```
array([[1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [1., 1., 1., 1.]])
```

`np.full`

Tạo một mảng với shape cho trước và khởi tạo bằng giá trị cụ thể. Đây là ma trận 3x4 chứa đầy giá trị `π`.

```
np.full((3,4), np.pi)
```

```
array([[3.14159265, 3.14159265, 3.14159265, 3.14159265],
       [3.14159265, 3.14159265, 3.14159265, 3.14159265],
       [3.14159265, 3.14159265, 3.14159265, 3.14159265]])
```

`np.empty`

Một mảng 2x3 chưa khởi tạo (nội dung của nó không thể dự đoán trước, vì nó là bất cứ giá trị nào đang có trong bộ nhớ tại thời đi

```
np.empty((2,3))
```

```
array([[-1.28822975e-231, -3.11108662e+231,  1.48219694e-323],
       [ 0.00000000e+000,  0.00000000e+000,  4.17201348e-309]])
```

np.array

Tất nhiên, bạn có thể khởi tạo một `ndarray` bằng cách sử dụng mảng Python thông thường (list). Chỉ cần gọi hàm `array`:

```
np.array([[1,2,3,4], [10, 20, 30, 40]])
```

```
array([[ 1,  2,  3,  4],
       [10, 20, 30, 40]])
```

```
## `np.arange`
Bạn có thể tạo một `ndarray` bằng hàm `arange` của NumPy, hàm này tương tự như hàm `range` có sẵn của Python:
```

```
np.arange(1, 5)
```

```
array([1, 2, 3, 4])
```

It also works with floats:

```
np.arange(1.0, 5.0)
```

```
array([1., 2., 3., 4.])
```

Of course, you can provide a step parameter:

```
np.arange(1, 5, 0.5)
```

```
array([1. , 1.5, 2. , 2.5, 3. , 3.5, 4. , 4.5])
```

However, when dealing with floats, the exact number of elements in the array is not always predictable. For example, consider this:

```
print(np.arange(0, 5/3, 1/3)) # depending on floating point errors, the max value is 4/3 or 5/3.
print(np.arange(0, 5/3, 0.333333333))
print(np.arange(0, 5/3, 0.333333334))
```

```
[0.         0.33333333 0.66666667 1.         1.33333333 1.66666667]
[0.         0.33333333 0.66666667 1.         1.33333333 1.66666667]
[0.         0.33333333 0.66666667 1.         1.33333334]
```

`np.linspace`

For this reason, it is generally preferable to use the `linspace` function instead of `arange` when working with floats. The `linspace` function returns an array containing a specific number of points evenly distributed between two values (note that the maximum value is *included*, contrary to `arange`):

```
print(np.linspace(0, 5/3, 6))
```

```
[0.         0.33333333 0.66666667 1.         1.33333333 1.66666667]
```

`np.rand` and `np.randn`

A number of functions are available in NumPy's `random` module to create `ndarray`s initialized with random values. For example, here is a 3x4 matrix initialized with random floats between 0 and 1 (uniform distribution):

```
np.random.rand(3,4)
```

```
array([[0.31506811, 0.9962061 , 0.22576662, 0.90354811],
       [0.75222452, 0.76662349, 0.1099069 , 0.98423873],
       [0.83335318, 0.35623138, 0.67330209, 0.23273007]])
```

Here's a 3x4 matrix containing random floats sampled from a univariate normal distribution (Gaussian distribution) of mean 0 and variance 1:

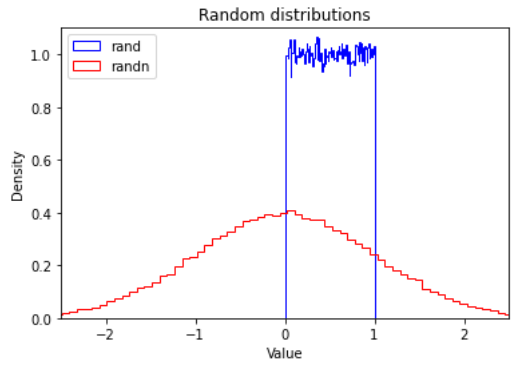
```
np.random.randn(3,4)
```

```
array([[ 1.56110261e+00,  2.43164383e-01, -7.35871062e-01,
         3.70101675e-04],
       [ 1.86890564e-01,  6.30621790e-01,  7.30944692e-01,
        -2.12995795e+00],
       [-5.61847627e-01, -8.63385596e-01, -3.25545246e-01,
         7.48875396e-01]])
```

To give you a feel of what these distributions look like, let's use matplotlib (see the matplotlib tutorial for more details):

```
import matplotlib.pyplot as plt
```

```
plt.hist(np.random.rand(100000), density=True, bins=100, histtype="step", color="blue", label="rand")
plt.hist(np.random.randn(100000), density=True, bins=100, histtype="step", color="red", label="randn")
plt.axis([-2.5, 2.5, 0, 1.1])
plt.legend(loc = "upper left")
plt.title("Random distributions")
plt.xlabel("Value")
plt.ylabel("Density")
plt.show()
```



np.fromfunction

You can also initialize an `ndarray` using a function:

```
def my_function(z, y, x):
    return x + 10 * y + 100 * z

np.fromfunction(my_function, (3, 2, 10))
```

```
array([[[  0.,   1.,   2.,   3.,   4.,   5.,   6.,   7.,   8.,   9.],
        [ 10.,  11.,  12.,  13.,  14.,  15.,  16.,  17.,  18.,  19.]],

       [[100., 101., 102., 103., 104., 105., 106., 107., 108., 109.],
        [110., 111., 112., 113., 114., 115., 116., 117., 118., 119.]],

       [[200., 201., 202., 203., 204., 205., 206., 207., 208., 209.],
        [210., 211., 212., 213., 214., 215., 216., 217., 218., 219.]]])
```

NumPy first creates three `ndarray`s (one per dimension), each of shape `(3, 2, 10)`. Each array has values equal to the coordinate along a specific axis. For example, all elements in the `z` array are equal to their z-coordinate:

```
[[[ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
  [ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]]

 [[ 1.  1.  1.  1.  1.  1.  1.  1.  1.  1.]
  [ 1.  1.  1.  1.  1.  1.  1.  1.  1.  1.]]

 [[ 2.  2.  2.  2.  2.  2.  2.  2.  2.  2.]
  [ 2.  2.  2.  2.  2.  2.  2.  2.  2.  2.]]]
```

So the terms `x`, `y` and `z` in the expression `x + 10 * y + 100 * z` above are in fact `ndarray`s (we will discuss arithmetic operations on arrays below). The point is that the function `my_function` is only called *once*, instead of once per element. This makes initialization very efficient.

Array data

`dtype`

NumPy's `ndarray`s are also efficient in part because all their elements must have the same type (usually numbers). You can check what the data type is by looking at the `dtype` attribute:

```
c = np.arange(1, 5)
print(c.dtype, c)
```

```
int64 [1 2 3 4]
```

```
c = np.arange(1.0, 5.0)
print(c.dtype, c)
```

```
float64 [1. 2. 3. 4.]
```

Instead of letting NumPy guess what data type to use, you can set it explicitly when creating an array by setting the `dtype` parameter:

```
d = np.arange(1, 5, dtype=np.complex64)
print(d.dtype, d)
```

```
complex64 [1.+0.j 2.+0.j 3.+0.j 4.+0.j]
```

Available data types include signed `int8`, `int16`, `int32`, `int64`, unsigned `uint8`|`16`|`32`|`64`, `float16`|`32`|`64` and `complex64`|`128`. Check out the documentation for the basic types and sized aliases for the full list.

`itemsize`

The `itemsize` attribute returns the size (in bytes) of each item:

```
e = np.arange(1, 5, dtype=np.complex64)
e.itemsize
```

```
8
```

`data` buffer

An array's data is actually stored in memory as a flat (one dimensional) byte buffer. It is available *via* the `data` attribute (you will rarely need it, though).

```
f = np.array([[1,2],[1000, 2000]], dtype=np.int32)
f.data
```

```
<memory at 0x7fb409690ad0>
```

In python 2, `f.data` is a buffer. In python 3, it is a memoryview.

```
if hasattr(f.data, "tobytes"):
    data_bytes = f.data.tobytes() # python 3
else:
    data_bytes = memoryview(f.data).tobytes() # python 2

data_bytes
```

```
b'\x01\x00\x00\x00\x02\x00\x00\x00\xe8\x03\x00\x00\xd0\x07\x00\x00'
```

Several `ndarray`s can share the same data buffer, meaning that modifying one will also modify the others. We will see an example in a minute.

Reshaping an array

In place

Changing the shape of an `ndarray` is as simple as setting its `shape` attribute. However, the array's size must remain the same.

```
g = np.arange(24)
print(g)
print("Rank:", g.ndim)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23]
Rank: 1
```

```
g.shape = (6, 4)
print(g)
print("Rank:", g.ndim)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]]
Rank: 2
```

```
g.shape = (2, 3, 4)
print(g)
print("Rank:", g.ndim)
```

```
[[[ 0  1  2  3]
  [ 4  5  6  7]
  [ 8  9 10 11]]

 [[12 13 14 15]
  [16 17 18 19]
  [20 21 22 23]]]
Rank: 3
```

`reshape`

The `reshape` function returns a new `ndarray` object pointing at the *same* data. This means that modifying one array will also modify the other.

```
g2 = g.reshape(4,6)
print(g2)
print("Rank:", g2.ndim)
```

```
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]
 [12 13 14 15 16 17]
 [18 19 20 21 22 23]]
Rank: 2
```

Set item at row 1, col 2 to 999 (more about indexing below).

```
g2[1, 2] = 999
g2
```

```
array([[  0,   1,   2,   3,   4,   5],
       [  6,   7, 999,   9,  10,  11],
       [ 12,  13,  14,  15,  16,  17],
       [ 18,  19,  20,  21,  22,  23]])
```

The corresponding element in `g` has been modified.

```
g
```

```
array([[[  0,   1,   2,   3],
        [  4,   5,   6,   7],
        [999,   9,  10,  11]],

       [[ 12,  13,  14,  15],
        [ 16,  17,  18,  19],
        [ 20,  21,  22,  23]]])
```

`ravel`

Finally, the `ravel` function returns a new one-dimensional `ndarray` that also points to the same data:

```
g.ravel()
```

```
array([  0,   1,   2,   3,   4,   5,   6,   7, 999,   9,  10,  11,  12,
        13,  14,  15,  16,  17,  18,  19,  20,  21,  22,  23])
```

Arithmetic operations

All the usual arithmetic operators (`+`, `-`, `*`, `/`, `//`, `**`, etc.) can be used with `ndarray`s. They apply *elementwise*:

```
a = np.array([14, 23, 32, 41])
b = np.array([5,  4,  3,  2])
print("a + b  =", a + b)
print("a - b  =", a - b)
print("a * b  =", a * b)
print("a / b  =", a / b)
print("a // b  =", a // b)
print("a % b  =", a % b)
print("a ** b =", a ** b)
```

```
a + b  = [19 27 35 43]
a - b  = [ 9 19 29 39]
a * b  = [70 92 96 82]
a / b  = [ 2.8         5.75       10.66666667 20.5       ]
a // b  = [ 2  5 10 20]
a % b  = [4 3 2 1]
a ** b = [537824 279841  32768   1681]
```

Note that the multiplication is *not* a matrix multiplication. We will discuss matrix operations below.

The arrays must have the same shape. If they do not, NumPy will apply the *broadcasting rules*.

Broadcasting

In general, when NumPy expects arrays of the same shape but finds that this is not the case, it applies the so-called *broadcasting* rules:

First rule

If the arrays do not have the same rank, then a 1 will be prepended to the smaller ranking arrays until their ranks match.

```
h = np.arange(5).reshape(1, 1, 5)
h
```

```
array([[[0, 1, 2, 3, 4]]])
```

Now let's try to add a 1D array of shape `(5,)` to this 3D array of shape `(1,1,5)`. Applying the first rule of broadcasting!

```
h + [10, 20, 30, 40, 50]  # same as: h + [[[10, 20, 30, 40, 50]]]
```

```
array([[[10, 21, 32, 43, 54]]])
```

Second rule

Arrays with a 1 along a particular dimension act as if they had the size of the array with the largest shape along that dimension. The value of the array element is repeated along that dimension.

```
k = np.arange(6).reshape(2, 3)
k
```

```
array([[0, 1, 2],
       [3, 4, 5]])
```

Let's try to add a 2D array of shape `(2,1)` to this 2D `ndarray` of shape `(2, 3)`. NumPy will apply the second rule of broadcasting:

```
k + [[100], [200]]  # same as: k + [[100, 100, 100], [200, 200, 200]]
```

```
array([[100, 101, 102],
       [203, 204, 205]])
```

Combining rules 1 & 2, we can do this:

```
k + [100, 200, 300]  # after rule 1: [[100, 200, 300]], and after rule 2: [[100, 200, 300], [100, 200, 300]]
```

```
array([[100, 201, 302],
       [103, 204, 305]])
```

And also, very simply:

```
k + 1000  # same as: k + [[1000, 1000, 1000], [1000, 1000, 1000]]
```

```
array([[1000, 1001, 1002],
       [1003, 1004, 1005]])
```

Third rule

After rules 1 & 2, the sizes of all arrays must match.

```
try:
    k + [33, 44]
except ValueError as e:
    print(e)
```

```
operands could not be broadcast together with shapes (2,3) (2,)
```

Broadcasting rules are used in many NumPy operations, not just arithmetic operations, as we will see below. For more details about broadcasting, check out the documentation.

Upcasting

When trying to combine arrays with different `dtype`s, NumPy will *upcast* to a type capable of handling all possible values (regardless of what the *actual* values are).

```
k1 = np.arange(0, 5, dtype=np.uint8)
print(k1.dtype, k1)
```

```
uint8 [0 1 2 3 4]
```

```
k2 = k1 + np.array([5, 6, 7, 8, 9], dtype=np.int8)
print(k2.dtype, k2)
```

```
int16 [ 5  7  9 11 13]
```

Note that `int16` is required to represent all *possible* `int8` and `uint8` values (from -128 to 255), even though in this case a `uint8` would have sufficed.

```
k3 = k1 + 1.5
print(k3.dtype, k3)
```

```
float64 [1.5 2.5 3.5 4.5 5.5]
```

Conditional operators

The conditional operators also apply elementwise:

```
m = np.array([20, -5, 30, 40])
m < [15, 16, 35, 36]
```

```
array([False,  True,  True, False])
```

And using broadcasting:

```
m < 25  # equivalent to m < [25, 25, 25, 25]
```

```
array([ True,  True, False, False])
```

This is most useful in conjunction with boolean indexing (discussed below).

```
m[m < 25]
```

```
array([20, -5])
```

Mathematical and statistical functions

Many mathematical and statistical functions are available for `ndarray`s.

`ndarray` methods

Some functions are simply `ndarray` methods, for example:

```
a = np.array([[-2.5, 3.1, 7], [10, 11, 12]])
print(a)
print("mean =", a.mean())
```

```
[[-2.5  3.1  7. ]
 [10.  11.  12. ]]
mean = 6.766666666666667
```

Note that this computes the mean of all elements in the `ndarray`, regardless of its shape.

Here are a few more useful `ndarray` methods:

```
for func in (a.min, a.max, a.sum, a.prod, a.std, a.var):
    print(func.__name__, "=", func())
```

```
min = -2.5
max = 12.0
sum = 40.6
prod = -71610.0
std = 5.084835843520964
var = 25.855555555555554
```

These functions accept an optional argument `axis` which lets you ask for the operation to be performed on elements along the given axis. For example:

```
c=np.arange(24).reshape(2,3,4)
c
```

```
array([[[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]],

       [[12, 13, 14, 15],
        [16, 17, 18, 19],
        [20, 21, 22, 23]]])
```

```
c.sum(axis=0)  # sum across matrices
```

```
array([[12, 14, 16, 18],
       [20, 22, 24, 26],
       [28, 30, 32, 34]])
```

```
c.sum(axis=1)  # sum across rows
```

```
array([[12, 15, 18, 21],
       [48, 51, 54, 57]])
```

You can also sum over multiple axes:

```
c.sum(axis=(0,2))  # sum across matrices and columns
```

```
array([ 60,  92, 124])
```

```
0+1+2+3 + 12+13+14+15, 4+5+6+7 + 16+17+18+19, 8+9+10+11 + 20+21+22+23
```

```
(60, 92, 124)
```

Universal functions

NumPy also provides fast elementwise functions called *universal functions*, or **ufunc**. They are vectorized wrappers of simple functions. For example `square` returns a new `ndarray` which is a copy of the original `ndarray` except that each element is squared:

```
a = np.array([[-2.5, 3.1, 7], [10, 11, 12]])
np.square(a)
```

```
array([[  6.25,   9.61,  49.  ],
       [100.  , 121.  , 144.  ]])
```

Here are a few more useful unary ufuncs:

```
print("Original ndarray")
print(a)
for func in (np.abs, np.sqrt, np.exp, np.log, np.sign, np.ceil, np.modf, np.isnan, np.cos):
    print("\n", func.__name__)
    print(func(a))
```

```
Original ndarray
[[-2.5  3.1  7. ]
 [10.  11.  12. ]]

 absolute
[[ 2.5  3.1  7. ]
 [10.  11.  12. ]]

 sqrt
[[       nan 1.76068169 2.64575131]
 [3.16227766 3.31662479 3.46410162]]

 exp
[[8.20849986e-02 2.21979513e+01 1.09663316e+03]
 [2.20264658e+04 5.98741417e+04 1.62754791e+05]]

 log
[[       nan 1.13140211 1.94591015]
 [2.30258509 2.39789527 2.48490665]]

 sign
[[-1.  1.  1.]
 [ 1.  1.  1.]]

 ceil
[[-2.  4.  7.]
 [10. 11. 12.]]

 modf
(array([[-0.5,  0.1,  0. ],
       [ 0. ,  0. ,  0. ]]), array([[-2.,  3.,  7.],
       [10., 11., 12.]]))

 isnan
[[False False False]
 [False False False]]

 cos
[[-0.80114362 -0.99913515  0.75390225]
 [-0.83907153  0.0044257   0.84385396]]
/var/folders/wy/h39t6kb11pnbb0pzhksd_fqh0000gq/T/ipykernel_88441/2634842825.py:5: RuntimeWarning: invalid value encountered in s
```

```
  print(func(a))
/var/folders/wy/h39t6kb11pnbb0pzhksd_fqh0000gq/T/ipykernel_88441/2634842825.py:5: RuntimeWarning: invalid value encountered in l
  print(func(a))
```

The two warnings are due to the fact that `sqrt()` and `log()` are undefined for negative numbers, which is why there is a `np.nan` value in the first cell of the output of these two functions.

Binary ufuncs

There are also many binary ufuncs, that apply elementwise on two `ndarray`s. Broadcasting rules are applied if the arrays do not have the same shape:

```
a = np.array([1, -2, 3, 4])
b = np.array([2, 8, -1, 7])
np.add(a, b)  # equivalent to a + b
```

```
array([ 3,  6,  2, 11])
```

```
np.greater(a, b)  # equivalent to a > b
```

```
array([False, False,  True, False])
```

```
np.maximum(a, b)
```

```
array([2, 8, 3, 7])
```

```
np.copysign(a, b)
```

```
array([ 1.,  2., -3.,  4.])
```

Array indexing

One-dimensional arrays

One-dimensional NumPy arrays can be accessed more or less like regular python arrays:

```
a = np.array([1, 5, 3, 19, 13, 7, 3])
a[3]
```

```
19
```

```
a[2:5]
```

```
array([ 3, 19, 13])
```

```
a[2:-1]
```

```
array([ 3, 19, 13,  7])
```

```
a[:2]
```

```
array([1, 5])
```

```
a[2::2]
```

```
array([ 3, 13,  3])
```

```
a[::-1]
```

```
array([ 3,  7, 13, 19,  3,  5,  1])
```

Of course, you can modify elements:

```
```

```
a[3]=999
a
```

```
array([  1,   5,   3, 999,  13,   7,   3])
```

You can also modify an `ndarray` slice:

```
a[2:5] = [997, 998, 999]
a
```

```
array([  1,   5, 997, 998, 999,   7,   3])
```

Differences with regular python arrays

Contrary to regular python arrays, if you assign a single value to an `ndarray` slice, it is copied across the whole slice, thanks to broadcasting rules discussed above.

```
a[2:5] = -1
a
```

```
array([ 1,  5, -1, -1, -1,  7,  3])
```

Also, you cannot grow or shrink `ndarray`s this way:

```
try:
    a[2:5] = [1,2,3,4,5,6]  # too long
except ValueError as e:
    print(e)
```

```
cannot copy sequence with size 6 to array axis with dimension 3
```

You cannot delete elements either:

```
try:
    del a[2:5]
except ValueError as e:
    print(e)
```

```
cannot delete array elements
```

Last but not least, `ndarray` **slices are actually *views*** on the same data buffer. This means that if you create a slice and modify it, you are actually going to modify the original `ndarray` as well!

```
a_slice = a[2:6]
a_slice[1] = 1000
a  # the original array was modified!
```

```
array([   1,    5,   -1, 1000,   -1,    7,    3])
```

```
a[3] = 2000
a_slice  # similarly, modifying the original array modifies the slice!
```

```
array([  -1, 2000,   -1,    7])
```

If you want a copy of the data, you need to use the `copy` method:

```
another_slice = a[2:6].copy()
another_slice[1] = 3000
a  # the original array is untouched
```

```
array([   1,    5,   -1, 2000,   -1,    7,    3])
```

```
a[3] = 4000
another_slice  # similarly, modifying the original array does not affect the slice copy
```

```
array([  -1, 3000,   -1,    7])
```

Multidimensional arrays

Multidimensional arrays can be accessed in a similar way by providing an index or slice for each axis, separated by commas:

```
b = np.arange(48).reshape(4, 12)
b
```

```
array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11],
       [12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23],
       [24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35],
       [36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47]])
```

```
b[1, 2]  # row 1, col 2
```

```
14
```

```
b[1, :]  # row 1, all columns
```

```
array([12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23])
```

```
b[:, 1]  # all rows, column 1
```

```
array([ 1, 13, 25, 37])
```

Caution: note the subtle difference between these two expressions:

```
b[1, :]
```

```
array([12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23])
```

```
b[1:2, :]
```

```
array([[12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23]])
```

The first expression returns row 1 as a 1D array of shape `(12,)`, while the second returns that same row as a 2D array of shape `(1, 12)`.

Fancy indexing

You may also specify a list of indices that you are interested in. This is referred to as *fancy indexing*.

```
b[(0,2), 2:5]  # rows 0 and 2, columns 2 to 4 (5-1)
```

```
array([[ 2,  3,  4],
       [26, 27, 28]])
```

```
b[:, (-1, 2, -1)]  # all rows, columns -1 (last), 2 and -1 (again, and in this order)
```

```
array([[11,  2, 11],
       [23, 14, 23],
       [35, 26, 35],
       [47, 38, 47]])
```

If you provide multiple index arrays, you get a 1D `ndarray` containing the values of the elements at the specified coordinates.

```
b[(-1, 2, -1, 2), (5, 9, 1, 9)]  # returns a 1D array with b[-1, 5], b[2, 9], b[-1, 1] and b[2, 9] (again)
```

```
array([41, 33, 37, 33])
```

Higher dimensions

Everything works just as well with higher dimensional arrays, but it's useful to look at a few examples:

```
c = b.reshape(4,2,6)
c
```

```
array([[[ 0,  1,  2,  3,  4,  5],
        [ 6,  7,  8,  9, 10, 11]],

       [[12, 13, 14, 15, 16, 17],
        [18, 19, 20, 21, 22, 23]],

       [[24, 25, 26, 27, 28, 29],
        [30, 31, 32, 33, 34, 35]],

       [[36, 37, 38, 39, 40, 41],
        [42, 43, 44, 45, 46, 47]]])
```

```
c[2, 1, 4]  # matrix 2, row 1, col 4
```

```
34
```

```
c[2, :, 3]  # matrix 2, all rows, col 3
```

```
array([27, 33])
```

If you omit coordinates for some axes, then all elements in these axes are returned:

```
c[2, 1]  # Return matrix 2, row 1, all columns.  This is equivalent to c[2, 1, :]
```

```
array([30, 31, 32, 33, 34, 35])
```

Ellipsis (`...`)

You may also write an ellipsis (`...`) to ask that all non-specified axes be entirely included.

```
c[2, ...]  #  matrix 2, all rows, all columns.  This is equivalent to c[2, :, :]
```

```
array([[24, 25, 26, 27, 28, 29],
       [30, 31, 32, 33, 34, 35]])
```

```
c[2, 1, ...]  # matrix 2, row 1, all columns.  This is equivalent to c[2, 1, :]
```

```
array([30, 31, 32, 33, 34, 35])
```

```
c[2, ..., 3]  # matrix 2, all rows, column 3.  This is equivalent to c[2, :, 3]
```

```
array([27, 33])
```

```
c[..., 3]  # all matrices, all rows, column 3.  This is equivalent to c[:, :, 3]
```

```
array([[ 3,  9],
       [15, 21],
       [27, 33],
       [39, 45]])
```

Boolean indexing

You can also provide an `ndarray` of boolean values on one axis to specify the indices that you want to access.

```
b = np.arange(48).reshape(4, 12)
b
```

```
array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11],
       [12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23],
       [24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35],
       [36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47]])
```

```
rows_on = np.array([True, False, True, False])
b[rows_on, :]  # Rows 0 and 2, all columns. Equivalent to b[(0, 2), :]
```

```
array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11],
       [24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35]])
```

```
cols_on = np.array([False, True, False] * 4)
b[:, cols_on]  # All rows, columns 1, 4, 7 and 10
```

```
array([[ 1,  4,  7, 10],
       [13, 16, 19, 22],
       [25, 28, 31, 34],
       [37, 40, 43, 46]])
```

`np.ix_`

You cannot use boolean indexing this way on multiple axes, but you can work around this by using the `ix_` function:

```
b[np.ix_(rows_on, cols_on)]
```

```
array([[ 1,  4,  7, 10],
       [25, 28, 31, 34]])
```

```
np.ix_(rows_on, cols_on)
```

```
(array([[0],
        [2]]),
 array([[ 1,  4,  7, 10]]))
```

If you use a boolean array that has the same shape as the `ndarray`, then you get in return a 1D array containing all the values that have `True` at their coordinate. This is generally used along with conditional operators:

```
b[b % 3 == 1]
```

```
array([ 1,  4,  7, 10, 13, 16, 19, 22, 25, 28, 31, 34, 37, 40, 43, 46])
```

Iterating

Iterating over `ndarray`s is very similar to iterating over regular python arrays. Note that iterating over multidimensional arrays is done with respect to the first axis.

```
c = np.arange(24).reshape(2, 3, 4)  # A 3D array (composed of two 3x4 matrices)
c
```

```
array([[[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]],

       [[12, 13, 14, 15],
        [16, 17, 18, 19],
        [20, 21, 22, 23]]])
```

```
for m in c:
    print("Item:")
    print(m)
```

```
Item:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
Item:
[[12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]]
```

```
for i in range(len(c)):  # Note that len(c) == c.shape[0]
    print("Item:")
    print(c[i])
```

```
Item:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
Item:
[[12 13 14 15]
```

```
 [16 17 18 19]
 [20 21 22 23]]
```

If you want to iterate on *all* elements in the `ndarray`, simply iterate over the `flat` attribute:

```
for i in c.flat:
    print("Item:", i)
```

```
Item: 0
Item: 1
Item: 2
Item: 3
Item: 4
Item: 5
Item: 6
Item: 7
Item: 8
Item: 9
Item: 10
Item: 11
Item: 12
Item: 13
Item: 14
Item: 15
Item: 16
Item: 17
Item: 18
Item: 19
Item: 20
Item: 21
Item: 22
Item: 23
```

Stacking arrays

It is often useful to stack together different arrays. NumPy offers several functions to do just that. Let's start by creating a few arrays.

```
q1 = np.full((3,4), 1.0)
q1
```

```
array([[1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [1., 1., 1., 1.]])
```

```
q2 = np.full((4,4), 2.0)
q2
```

```
array([[2., 2., 2., 2.],
       [2., 2., 2., 2.],
       [2., 2., 2., 2.],
       [2., 2., 2., 2.]])
```

```
q3 = np.full((3,4), 3.0)
q3
```

```
array([[3., 3., 3., 3.],
       [3., 3., 3., 3.],
       [3., 3., 3., 3.]])
```

`vstack`

Now let's stack them vertically using `vstack`:

```
q4 = np.vstack((q1, q2, q3))
q4
```

```
array([[1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [2., 2., 2., 2.],
       [2., 2., 2., 2.],
       [2., 2., 2., 2.],
       [2., 2., 2., 2.],
```

```
       [3., 3., 3., 3.],
       [3., 3., 3., 3.],
       [3., 3., 3., 3.]])
```

```
q4.shape
```

```
(10, 4)
```

It was possible because q1, q2 and q3 all have the same shape (except for the vertical axis, but that's ok since we are stacking on that axis).

`hstack`

We can also stack arrays horizontally using `hstack`:

```
q5 = np.hstack((q1, q3))
q5
```

```
array([[1., 1., 1., 1., 3., 3., 3., 3.],
       [1., 1., 1., 1., 3., 3., 3., 3.],
       [1., 1., 1., 1., 3., 3., 3., 3.]])
```

```
q5.shape
```

```
(3, 8)
```

It is possible because q1 and q3 both have 3 rows. But since q2 has 4 rows, it cannot be stacked horizontally with q1 and q3:

```
try:
    q5 = np.hstack((q1, q2, q3))
except ValueError as e:
    print(e)
```

```
all the input array dimensions for the concatenation axis must match exactly, but along dimension 0, the array at index 0 has si
```

`concatenate`

The `concatenate` function stacks arrays along any given existing axis.

```
q7 = np.concatenate((q1, q2, q3), axis=0)  # Equivalent to vstack
q7
```

```
array([[1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [2., 2., 2., 2.],
       [2., 2., 2., 2.],
       [2., 2., 2., 2.],
       [2., 2., 2., 2.],
       [3., 3., 3., 3.],
       [3., 3., 3., 3.],
       [3., 3., 3., 3.]])
```

```
q7.shape
```

```
(10, 4)
```

As you might guess, `hstack` is equivalent to calling `concatenate` with `axis=1`.

`stack`

The `stack` function stacks arrays along a new axis. All arrays have to have the same shape.

```
q8 = np.stack((q1, q3))
q8
```

```
array([[[1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.]],

       [[3., 3., 3., 3.],
        [3., 3., 3., 3.],
        [3., 3., 3., 3.]]])
```

```
q8.shape
```

```
(2, 3, 4)
```

Splitting arrays

Splitting is the opposite of stacking. For example, let's use the `vsplit` function to split a matrix vertically.

First let's create a 6x4 matrix:

```
r = np.arange(24).reshape(6,4)
r
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [12, 13, 14, 15],
       [16, 17, 18, 19],
       [20, 21, 22, 23]])
```

Now let's split it in three equal parts, vertically:

```
r1, r2, r3 = np.vsplit(r, 3)
r1
```

```
array([[0, 1, 2, 3],
       [4, 5, 6, 7]])
```

```
r2
```

```
array([[ 8,  9, 10, 11],
       [12, 13, 14, 15]])
```

```
r3
```

```
array([[16, 17, 18, 19],
       [20, 21, 22, 23]])
```

There is also a `split` function which splits an array along any given axis. Calling `vsplit` is equivalent to calling `split` with `axis=0`. There is also an `hsplit` function, equivalent to calling `split` with `axis=1`:

```
r4, r5 = np.hsplit(r, 2)
r4
```

```
array([[ 0,  1],
       [ 4,  5],
       [ 8,  9],
       [12, 13],
       [16, 17],
       [20, 21]])
```

```
r5
```

```
array([[ 2,  3],
       [ 6,  7],
       [10, 11],
       [14, 15],
       [18, 19],
       [22, 23]])
```

Transposing arrays

The `transpose` method creates a new view on an `ndarray`'s data, with axes permuted in the given order.

For example, let's create a 3D array:

```
t = np.arange(24).reshape(4,2,3)
t
```

```
array([[[ 0,  1,  2],
        [ 3,  4,  5]],

       [[ 6,  7,  8],
        [ 9, 10, 11]],

       [[12, 13, 14],
        [15, 16, 17]],

       [[18, 19, 20],
        [21, 22, 23]]])
```

Now let's create an `ndarray` such that the axes `0, 1, 2` (depth, height, width) are re-ordered to `1, 2, 0` (depth→width, height→depth, width→height):

```
t1 = t.transpose((1,2,0))
t1
```

```
array([[[ 0,  6, 12, 18],
        [ 1,  7, 13, 19],
        [ 2,  8, 14, 20]],

       [[ 3,  9, 15, 21],
        [ 4, 10, 16, 22],
        [ 5, 11, 17, 23]]])
```

```
t1.shape
```

```
(2, 3, 4)
```

By default, `transpose` reverses the order of the dimensions:

```
t2 = t.transpose()  # equivalent to t.transpose((2, 1, 0))
t2
```

```
array([[[ 0,  6, 12, 18],
        [ 3,  9, 15, 21]],

       [[ 1,  7, 13, 19],
        [ 4, 10, 16, 22]],

       [[ 2,  8, 14, 20],
        [ 5, 11, 17, 23]]])
```

```
t2.shape
```

```
(3, 2, 4)
```

NumPy provides a convenience function `swapaxes` to swap two axes. For example, let's create a new view of `t` with depth and height swapped:

```
t3 = t.swapaxes(0,1)  # equivalent to t.transpose((1, 0, 2))
t3
```

```
array([[[ 0,  1,  2],
        [ 6,  7,  8],
        [12, 13, 14],
        [18, 19, 20]],

       [[ 3,  4,  5],
        [ 9, 10, 11],
        [15, 16, 17],
        [21, 22, 23]]])
```

```
t3.shape
```

```
(2, 4, 3)
```

Linear algebra

NumPy 2D arrays can be used to represent matrices efficiently in python. We will just quickly go through some of the main matrix operations available. For more details about Linear Algebra, vectors and matrices, go through the Linear Algebra tutorial.

Matrix transpose

The `T` attribute is equivalent to calling `transpose()` when the rank is ≥2:

```
m1 = np.arange(10).reshape(2,5)
m1
```

```
array([[0, 1, 2, 3, 4],
       [5, 6, 7, 8, 9]])
```

```
m1.T
```

```
array([[0, 5],
       [1, 6],
       [2, 7],
       [3, 8],
       [4, 9]])
```

The `T` attribute has no effect on rank 0 (empty) or rank 1 arrays:

```
m2 = np.arange(5)
m2
```

```
array([0, 1, 2, 3, 4])
```

```
m2.T
```

```
array([0, 1, 2, 3, 4])
```

We can get the desired transposition by first reshaping the 1D array to a single-row matrix (2D):

```
m2r = m2.reshape(1,5)
m2r
```

```
array([[0, 1, 2, 3, 4]])
```

```
m2r.T
```

```
array([[0],
       [1],
       [2],
       [3],
       [4]])
```

Matrix multiplication

Let's create two matrices and execute a matrix multiplication using the `dot()` method.

```
n1 = np.arange(10).reshape(2, 5)
n1
```

```
array([[0, 1, 2, 3, 4],
       [5, 6, 7, 8, 9]])
```

```
n2 = np.arange(15).reshape(5,3)
n2
```

```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
```

```
        [ 9, 10, 11],
        [12, 13, 14]])
```

```
n1.dot(n2)
```

```
array([[ 90, 100, 110],
       [240, 275, 310]])
```

Caution: as mentioned previously, `n1*n2` is *not* a matrix multiplication, it is an elementwise product (also called a Hadamard product).

Matrix inverse and pseudo-inverse

Many of the linear algebra functions are available in the `numpy.linalg` module, in particular the `inv` function to compute a square matrix's inverse:

```
import numpy.linalg as linalg

m3 = np.array([[1,2,3],[5,7,11],[21,29,31]])
m3
```

```
array([[ 1,  2,  3],
       [ 5,  7, 11],
       [21, 29, 31]])
```

```
linalg.inv(m3)
```

```
array([[-2.31818182,  0.56818182,  0.02272727],
       [ 1.72727273, -0.72727273,  0.09090909],
       [-0.04545455,  0.29545455, -0.06818182]])
```

You can also compute the pseudoinverse using `pinv`:

```
linalg.pinv(m3)
```

```
array([[-2.31818182,  0.56818182,  0.02272727],
       [ 1.72727273, -0.72727273,  0.09090909],
       [-0.04545455,  0.29545455, -0.06818182]])
```

Identity matrix

The product of a matrix by its inverse returns the identity matrix (with small floating point errors):

```
m3.dot(linalg.inv(m3))
```

```
array([[ 1.00000000e+00, -1.66533454e-16,  0.00000000e+00],
       [ 6.31439345e-16,  1.00000000e+00, -1.38777878e-16],
       [ 5.21110932e-15, -2.38697950e-15,  1.00000000e+00]])
```

You can create an identity matrix of size NxN by calling `eye(N)` function:

```
np.eye(3)
```

```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

QR decomposition

The `qr` function computes the QR decomposition of a matrix:

```
q, r = linalg.qr(m3)
q
```

```
array([[-0.04627448,  0.98786672,  0.14824986],
       [-0.23137241,  0.13377362, -0.96362411],
       [-0.97176411, -0.07889213,  0.22237479]])
```

```
r
```

```
array([[-21.61018278, -29.89331494, -32.80860727],
       [  0.        ,   0.62427688,   1.9894538 ],
       [  0.        ,   0.        ,  -3.26149699]])
```

```
q.dot(r)  # q.r equals m3
```

```
array([[ 1.,  2.,  3.],
       [ 5.,  7., 11.],
       [21., 29., 31.]])
```

Determinant

The `det` function computes the matrix determinant:

```
linalg.det(m3)  # Computes the matrix determinant
```

```
43.99999999999997
```

Eigenvalues and eigenvectors

The `eig` function computes the eigenvalues and eigenvectors of a square matrix:

```
eigenvalues, eigenvectors = linalg.eig(m3)
eigenvalues # λ
```

```
array([42.26600592, -0.35798416, -2.90802176])
```

```
eigenvectors # v
```

```
array([[-0.08381182, -0.76283526, -0.18913107],
       [-0.3075286 ,  0.64133975, -0.6853186 ],
       [-0.94784057, -0.08225377,  0.70325518]])
```

```
m3.dot(eigenvectors) - eigenvalues * eigenvectors  # m3.v - λ*v = 0
```

```
array([[ 6.66133815e-15,  1.66533454e-15, -3.10862447e-15],
       [ 7.10542736e-15,  5.16253706e-15, -5.32907052e-15],
       [ 3.55271368e-14,  4.94743135e-15, -9.76996262e-15]])
```

Singular Value Decomposition

The `svd` function takes a matrix and returns its singular value decomposition:

```
m4 = np.array([[1,0,0,0,2], [0,0,3,0,0], [0,0,0,0,0], [0,2,0,0,0]])
m4
```

```
array([[1, 0, 0, 0, 2],
       [0, 0, 3, 0, 0],
       [0, 0, 0, 0, 0],
       [0, 2, 0, 0, 0]])
```

```
U, S_diag, V = linalg.svd(m4)
U
```

```
array([[ 0.,  1.,  0.,  0.],
       [ 1.,  0.,  0.,  0.],
       [ 0.,  0.,  0., -1.],
       [ 0.,  0.,  1.,  0.]])
```

```
S_diag
```

```
array([3.        , 2.23606798, 2.        , 0.        ])
```

The `svd` function just returns the values in the diagonal of Σ, but we want the full Σ matrix, so let's create it:

```
S = np.zeros((4, 5))
S[np.diag_indices(4)] = S_diag
S  # Σ
```

```
array([[3.        , 0.        , 0.        , 0.        , 0.        ],
       [0.        , 2.23606798, 0.        , 0.        , 0.        ],
       [0.        , 0.        , 2.        , 0.        , 0.        ],
       [0.        , 0.        , 0.        , 0.        , 0.        ]])
```

```
V
```

```
array([[-0.        ,  0.        ,  1.        , -0.        ,  0.        ],
       [ 0.4472136 ,  0.        ,  0.        ,  0.        ,  0.89442719],
       [-0.        ,  1.        ,  0.        , -0.        ,  0.        ],
       [ 0.        ,  0.        ,  0.        ,  1.        ,  0.        ],
       [-0.89442719,  0.        ,  0.        ,  0.        ,  0.4472136 ]])
```

```
U.dot(S).dot(V) # U.Σ.V == m4
```

```
array([[1., 0., 0., 0., 2.],
       [0., 0., 3., 0., 0.],
       [0., 0., 0., 0., 0.],
       [0., 2., 0., 0., 0.]])
```

Diagonal and trace

```
np.diag(m3)  # the values in the diagonal of m3 (top left to bottom right)
```

```
array([ 1,  7, 31])
```

```
m3.trace()  # equivalent to np.diag(m3).sum()
```

```
39
```

Solving a system of linear scalar equations

The `solve` function solves a system of linear scalar equations, such as:

- $2x + 6y = 6$
- $5x + 3y = -9$

```
coeffs  = np.array([[2, 6], [5, 3]])
depvars = np.array([6, -9])
solution = linalg.solve(coeffs, depvars)
solution
```

```
array([-3.,  2.])
```

Let's check the solution:

```
coeffs.dot(solution), depvars  # yep, it's the same
```

```
(array([ 6., -9.]), array([ 6, -9]))
```

Looks good! Another way to check the solution:

```
np.allclose(coeffs.dot(solution), depvars)
```

```
True
```

Vectorization

Instead of executing operations on individual array items, one at a time, your code is much more efficient if you try to stick to array operations. This is called *vectorization*. This way, you can benefit from NumPy's many optimizations.

For example, let's say we want to generate a 768x1024 array based on the formula $sin(xy/40.5)$. A **bad** option would be to do the math in python using nested loops:

```
import math

data = np.empty((768, 1024))
for y in range(768):
    for x in range(1024):
        data[y, x] = math.sin(x * y / 40.5)  # BAD! Very inefficient.
```

Sure, this works, but it's terribly inefficient since the loops are taking place in pure python. Let's vectorize this algorithm. First, we will use NumPy's `meshgrid` function which generates coordinate matrices from coordinate vectors.

```
x_coords = np.arange(0, 1024)  # [0, 1, 2, ..., 1023]
y_coords = np.arange(0, 768)   # [0, 1, 2, ..., 767]
X, Y = np.meshgrid(x_coords, y_coords)
X
```

```
array([[   0,    1,    2, ..., 1021, 1022, 1023],
       [   0,    1,    2, ..., 1021, 1022, 1023],
       [   0,    1,    2, ..., 1021, 1022, 1023],
       ...,
       [   0,    1,    2, ..., 1021, 1022, 1023],
       [   0,    1,    2, ..., 1021, 1022, 1023],
       [   0,    1,    2, ..., 1021, 1022, 1023]])
```

```
Y
```

```
array([[  0,   0,   0, ...,   0,   0,   0],
       [  1,   1,   1, ...,   1,   1,   1],
       [  2,   2,   2, ...,   2,   2,   2],
       ...,
       [765, 765, 765, ..., 765, 765, 765],
       [766, 766, 766, ..., 766, 766, 766],
       [767, 767, 767, ..., 767, 767, 767]])
```

As you can see, both `X` and `Y` are 768x1024 arrays, and all values in `X` correspond to the horizontal coordinate, while all values in `Y` correspond to the vertical coordinate.

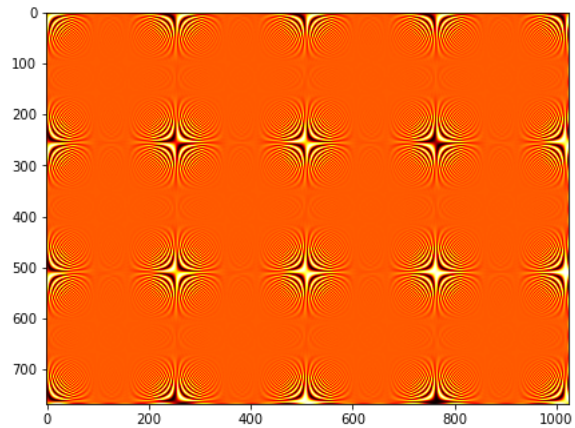
Now we can simply compute the result using array operations:

```
data = np.sin(X * Y / 40.5)
```

Now we can plot this data using matplotlib's `imshow` function (see the matplotlib tutorial).

```
import matplotlib.pyplot as plt

fig = plt.figure(1, figsize=(7, 6))
plt.imshow(data, cmap="hot")
plt.show()
```



Saving and loading

NumPy makes it easy to save and load `ndarray`s in binary or text format.

Binary `.npy` format

Let's create a random array and save it.

```
a = np.random.rand(2,3)
a
```

```
array([[0.61340716, 0.6785629 , 0.07091271],
       [0.60611491, 0.76189645, 0.2167531 ]])
```

```
np.save("my_array", a)
```

Done! Since the file name contains no file extension, NumPy automatically added `.npy`. Let's take a peek at the file content:

```
with open("my_array.npy", "rb") as f:
    content = f.read()

content
```

```
b"\x93NUMPY\x01\x00v\x00{'descr': '<f8', 'fortran_order': False, 'shape': (2, 3), }
\nEB\x17\x0e\x08\xa1\xe3?O^\xcc\x8b\xc9\xb6\xe5?\x90(\r\xc6U'\xb2?r\xcf\n\x19Ke\xe3?\xf8/q\xaata\xe8?H#\xae\xbf\x90\xbe\xcb?"
```

To load this file into a NumPy array, simply call `load`:

```
a_loaded = np.load("my_array.npy")
a_loaded
```

```
array([[0.61340716, 0.6785629 , 0.07091271],
       [0.60611491, 0.76189645, 0.2167531 ]])
```

Text format

Let's try saving the array in text format:

```
np.savetxt("my_array.csv", a)
```

Now let's look at the file content:

```
with open("my_array.csv", "rt") as f:
    print(f.read())
```

```
6.134071612560282327e-01 6.785629015384503360e-01 7.091270528067838974e-02
6.061149109941494917e-01 7.618964508960734960e-01 2.167530952395060329e-01
```

This is a CSV file with tabs as delimiters. You can set a different delimiter:

```
np.savetxt("my_array.csv", a, delimiter=",")
```

To load this file, just use `loadtxt`:

```
a_loaded = np.loadtxt("my_array.csv", delimiter=",")
a_loaded
```

```
array([[0.61340716, 0.6785629 , 0.07091271],
       [0.60611491, 0.76189645, 0.2167531 ]])
```

Zipped `.npz` format

It is also possible to save multiple arrays in one zipped file:

```
b = np.arange(24, dtype=np.uint8).reshape(2, 3, 4)
b
```

```
array([[[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]],

       [[12, 13, 14, 15],
        [16, 17, 18, 19],
        [20, 21, 22, 23]]], dtype=uint8)
```

```
np.savez("my_arrays", my_a=a, my_b=b)
```

Again, let's take a peek at the file content. Note that the `.npz` file extension was automatically added.

```
with open("my_arrays.npz", "rb") as f:
    content = f.read()

repr(content)[:180] + "[...]"
```

```
'b"PK\\x03\\x04\\x14\\x00\\x00\\x00\\x00\\x00\\x00\\x00\\x00!\\x00\\xe5\\x92\\xa1\\xfc\\xb0\\x00\\x00\\x00\\xb0\\x00\\x00\\x00\\x08\\
```

You then load this file like so:

```
my_arrays = np.load("my_arrays.npz")
my_arrays
```

```
<numpy.lib.npyio.NpzFile at 0x7fb3d823fa90>
```

This is a dict-like object which loads the arrays lazily:

```
my_arrays.keys()
```

```
KeysView(<numpy.lib.npyio.NpzFile object at 0x7fb3d823fa90>)
```

```
my_arrays["my_a"]
```

```
array([[0.61340716, 0.6785629 , 0.07091271],
       [0.60611491, 0.76189645, 0.2167531 ]])
```

What's next?

Now you know all the fundamentals of NumPy, but there are many more options available. The best way to learn more is to experiment with NumPy, and go through the excellent reference documentation to find more functions and features you may be interested in.